

엣지 실리콘에서 스트리밍 VLM 양자화

4B 스트리밍 VLM을 소비자용 맥에 올리기: 실측 footprint, 지연, 패키징

2i

2026-07-29

초록

4B급 스트리밍 비전-언어모델(VLM) 하나를 데이터센터 GPU가 아니라 소비자용 Apple 실리콘 Mac 한 대에 올려 실행했다. 대상은 Apache-2.0 라이선스로 공개된 microsoft/Mage-VL이며, Mage-ViT 비전 인코더와 Qwen3-4B-Instruct-2507 언어 백본을 결합한 구조다. Apple M4 Pro(통합 메모리 48GB) 위에서 bf16, INT8 weight-only 양자화, INT4 weight-only 양자화 세 가지 정밀도로 이미지 이해 추론을 실제로 돌리고 디바이스 메모리, 디코드 처리량, 첫 토큰까지 걸리는 시간, 답변 품질을 측정했다. 결과는 두 갈래로 갈린다. 첫째, 양자화는 디바이스 메모리를 bf16의 9.49GB에서 INT8 5.86GB(38% 감소), INT4 4.15GB (56% 감소)로 확실히 줄였고, 온디스크 크기도 9.8GB에서 커뮤니티 MLX 4비트 체크포인트 기준 2.9GB까지 줄어드는 여지를 보였다. 답변 품질도 세 정밀도 모두에서 합성 테스트 이미지의 핵심 내용을 정확히 읽어냈다. 둘째, 그런데 같은 양자화가 디코드 속도는 정반대로 움직였다. transformers + Metal(MPS) 경로에서 디코드 처리량은 bf16 13.01 tok/s에서 INT8 2.31 tok/s, INT4 0.47 tok/s로 오히려 느려졌다. 메모리를 줄이는 것과 속도를 올리는 것은 서로 다른 축이며, 저비트 행렬곱을 위한 융합 커널이 없는 런타임에서는 양자화가 메모리 절감만 주고 속도는 감아야 할 빛으로 돌아온다는 것이 이 논문의 핵심 발견이다. 스트리밍 비디오 경로(DCVC-RT 신경 코덱)는 CUDA 확장에 묶여 있어 Apple Metal에서는 애초에 시도하지 않았다. 이 경계는 결합이 아니라 다음 작업(MLX 아키텍처 포팅)을 정의하는 스코프다. 2i는 모델을 파는 것이 아니라 이렇게 측정된 패키징 작업, 즉 양자화 선택과 런타임 선택과 스코프 정의를 판다.

목차

1 요약	3
2 배경	3
2.1 왜 엣지에서 VLM을 돌리려 하는가	3
2.2 모델 구조	3
2.3 저자 보고 수치 (참고, 우리 측정 아님)	3
3 방법	4
3.1 하드웨어와 스택	4
3.2 양자화 방법	4
3.3 테스트 입력	4
3.4 측정 지표	4
4 결과	4
4.1 측정표	4
4.2 그림: 메모리와 처리량	5
5 해석: 메모리는 줄고 속도도 줄어드는 이중성	6
6 한계와 스코프	6
7 데이터와 재현	6
8 참고문헌	7

1 요약

4B급 스트리밍 비전-언어모델(VLM) 하나를 데이터센터 GPU가 아니라 소비자용 Apple 실리콘 Mac 한 대에 올려 실행했다. 대상은 Apache-2.0 라이선스로 공개된 microsoft/Mage-VL이며, Mage-ViT 비전 인코더와 Qwen3-4B-Instruct-2507 언어 백본을 결합한 구조다. Apple M4 Pro(통합 메모리 48GB) 위에서 bf16, INT8 weight-only 양자화, INT4 weight-only 양자화 세 가지 정밀도로 이미지 이해 추론을 실제로 돌리고 디바이스 메모리, 디코드 처리량, 첫 토큰까지 걸리는 시간, 답변 품질을 측정했다. 결과는 두 갈래로 갈린다. 첫째, 양자화는 디바이스 메모리를 bf16의 9.49GB에서 INT8 5.86GB(38% 감소), INT4 4.15GB(56% 감소)로 확실히 줄였고, 온디스크 크기도 9.8GB에서 커뮤니티 MLX 4비트 체크포인트 기준 2.9GB까지 줄어드는 여지를 보였다. 답변 품질도 세 정밀도 모두에서 합성 테스트 이미지의 핵심 내용을 정확히 읽어냈다. 둘째, 그런데 같은 양자화가 디코드 속도는 정반대로 움직였다. transformers + Metal(MPS) 경로에서 디코드 처리량은 bf16 13.01 tok/s에서 INT8 2.31 tok/s, INT4 0.47 tok/s로 오히려 느려졌다. 메모리를 줄이는 것과 속도를 올리는 것은 서로 다른 축이며, 저비트 행렬곱을 위한 융합 커널이 없는 런타임에서는 양자화가 메모리 절감만 주고 속도는 갇아야 할 빗으로 돌아온다는 것이 이 논문의 핵심 발견이다. 스트리밍 비디오 경로(DCVC-RT 신경 코덱)는 CUDA 확장에 묶여 있어 Apple Metal에서는 애초에 시도하지 않았다. 이 경계는 결합이 아니라 다음 작업(MLX 아키텍처 포팅)을 정의하는 스코프다. 2i는 모델을 파는 것이 아니라 이렇게 측정된 패키징 작업, 즉 양자화 선택과 런타임 선택과 스코프 정의를 판다.

2 배경

2.1 왜 엣지에서 VLM을 돌리려 하는가

매장 운영 현장의 CCTV, 순찰 로봇의 카메라, 제조 현장의 검사 라인처럼 카메라가 이미 있는 곳에 비전-언어모델을 엮으려는 수요는 꾸준히 늘고 있다. 문제는 이런 현장 대부분이 데이터센터 GPU를 돌 전력·공간·네트워크 예산이 없다는 점이다. 영상을 클라우드로 실시간 전송해 추론하는 구조는 지연과 대역폭, 그리고 보안·주권 요구사항(온프레임 처리) 앞에서 자주 막힌다. 그래서 질문은 단순해진다. SOTA 수준의 VLM을, 서버실이 아니라 매장 뒷방에 놓인 소비자용 컴퓨터 한 대에서 돌릴 수 있는가.

microsoft/Mage-VL은 이 질문에 답하기 좋은 대상이다. 이미지-텍스트 이해와 비디오/스트리밍 이해를 함께 지원하는 4B급 모델이면서, **Apache-2.0** 라이선스로 공개되어 있다. 이 라이선스가 이 논문의 존재 이유 중 하나다. 연구용 데모가 아니라 상업적으로 패키징해 고객사에 배치할 수 있는 모델이라는 뜻이고, 그래서 “이걸 실제로 소비자용 하드웨어에 올리면 무엇을 얻고 무엇을 내주는가”라는 질문이 사업적으로도 의미를 가진다.

2.2 모델 구조

Mage-VL은 두 부분으로 이루어진다. 시각 인코더는 자체 아키텍처인 Mage-ViT(4B 스케일)이고, 언어 백본은 Qwen3-4B-Instruct-2507이다. 우리가 실제로 로드해 확인한 파라미터 수는 **4.742B**다(공개 카드의 “4B” 표기와 합치한다). 모델은 여기에 두 가지를 더 얹는다. 하나는 “StreamMind”라는 이벤트 게이트로, 스트리밍 입력에서 능동적으로 주목할 순간을 트리거하기 위한 보조 가중치(1.07GB)다. 다른 하나는 DCVC-RT 신경 비디오 코덱으로, 비디오/스트리밍 경로의 프레임 처리를 가속하기 위해 C++/CUDA 확장으로 구현되어 있다.

여기서 이 논문의 스코프를 미리 밝혀 둔다. 이미지 한 장, 혹은 비디오의 개별 프레임을 이해하는 경로는 이 신경 코덱을 아예 거치지 않는다. CUDA가 전혀 필요 없다는 뜻이다. 반면 진짜 스트리밍 비디오의 빠른 경로는 DCVC-RT의 .cu CUDA 커널에 묶여 있고, 이는 Apple Metal에서 그대로 돌아가지 않는다. 그래서 이 논문은 이미지/프레임 이해 경로를 측정하고, 스트리밍 비디오 경로는 “지금은 안 된다”가 아니라 “포팅이 필요한 다음 작업”으로 명시적으로 분리해 다룬다. 이 구분을 흐리지 않는 것이 이 논문의 정직성 원칙이다.

2.3 저자 보고 수치 (참고, 우리 측정 아님)

Mage-VL 저자들은 8×B200 데이터센터 노드에서 벤치마크한 수치를 보고한다. 시각 토큰을 75% 이상 줄이고, 균일 프레임 샘플링 대비 최대 3.5배의 실제 체감(wall-clock) 속도 향상을 얻으며, 676개 토큰 기준으로 Food-101에서 96.1% 이상, ImageNet에서 86.3% 이상의 정확도를, SoccerNet 스트리밍 과제에서 F1 16.35·ROC-AUC 83.14를, OVO-Bench에서 64.00을 기록했다고 밝힌다. 이 수치들은 전부 **저자 보고**이며, 이 논문의 어떤 실행치와도 직접 비교하지 않는다. 우리가 이 논문에서 다루는 것은 완전히 다른 질문, 즉 “이 모델을 데이터센터가 아니라 소비자용 Mac에 올리면 무엇이 벌어지는가” 이다.

3 방법

3.1 하드웨어와 스택

측정 환경은 Apple M4 Pro, 통합 메모리 48GB, macOS, arm64 아키텍처의 일반 소비자용 컴퓨터(Mac mini급 사양)다. 데이터센터 GPU가 아니라는 점을 강조해 둔다. 소프트웨어 스택은 transformers 5.12와 torch 2.12이며, 연산 디바이스는 Apple Metal을 사용하는 MPS다.

3.2 양자화 방법

양자화는 optimum-quanto의 weight-only 경로를 사용해 언어모델 서브모듈(language_model)에만 적용했다. 시각 인코더(Mage-ViT)는 세 경로 모두에서 bf16으로 유지했다. 즉 우리가 바꾸는 것은 오직 언어 백본 가중치의 표현 정밀도(bf16 → INT8 → INT4)뿐이고, 시각 처리 경로는 고정해 변수를 하나로 줄였다.

이와 별도로, MLX 런타임에서 돌아가는 커뮤니티 제작 INT4 체크포인트를 mlx-vlm으로 로드해보았다. 이 시도는 결과 섹션에서 밝히듯 로드 자체가 실패했지만, 온디스크 크기라는 유의미한 데이터 포인트 하나를 남겼기에 결과에 포함한다.

3.3 테스트 입력

테스트 이미지는 **합성 이미지**다. 실제 사진이 아니라 우리가 직접 만든 768×512 크기의 “역 전광판” 장면으로, 정답(ground truth)을 우리가 정확히 알고 있다는 것이 핵심이다. 제목은 “CENTRAL STATION - DEPARTURES”이고, 아래 세 줄에 각각 “14:05 / PLATFORM 2 / ON TIME”, “14:20 / PLATFORM 5 / DELAYED”, “14:35 / PLATFORM 1 / BOARDING”이 적혀 있으며, 기차 픽토그램과 빨간 STOP 표지판이 함께 그려져 있다. 실제 사진을 쓰지 않은 이유는 정답을 완전히 통제해 “모델이 실제로 화면을 읽었는가, 아니면 그럴듯하게 지어냈는가”를 명확히 판정하기 위해서다. 이 점을 숨기지 않고 밝히는 것이 정직성 원칙의 일부다.

프롬프트는 “Describe this image in detail...” 형태의 상세 서술 요청이다. 비전 토큰을 포함한 프롬프트 길이는 419토큰이고, 매 실행마다 128토큰을 생성하도록 고정했다.

3.4 측정 지표

측정한 지표는 네 가지다. **디바이스 메모리**는 MPS가 실제로 할당한 메모리량이다. **디코드 처리량**은 초당 생성 토큰 수(tok/s)로, 128토큰 생성 구간의 평균이다. **첫 토큰까지 걸리는 시간**(time-to-first-token, TTFT)은 프롬프트를 다 읽고 첫 출력 토큰이 나오기까지의 지연이며 419토큰 프리필 구간의 비용을 반영한다. **품질**은 합성 이미지의 정답 요소(제목 문구, 3행의 시각·플랫폼·상태 정보, 기차 픽토그램, STOP 표지판)를 모델의 서술이 정확히 포함하는지를 육안으로 대조해 판정했다. 정확도 수치를 자동 스코어링한 것이 아니라, 이 하나의 합성 이미지에 대한 정성적 정답-대조라는 점을 분명히 한다.

4 결과

4.1 측정표

아래 표는 세 정밀도 경로와 MLX 시도 전부를 정리한 것이다. 모든 수치는 Apple M4 Pro에서 직접 측정한 실측치이며, MLX 행의 온디스크 크기만 예외로 커뮤니티 체크포인트의 공개된 파일 크기를 인용했다.

정밀도	런타임	디바이스 메모리 (MPS 할당)	디코드 tok/s	TTFT	품질 (합성 테스트 이미지)	온디스크
bf16	transformers + MPS	9.49 GB	13.01	2.94 s	정확함 (전광판 제목·구조를 정확히 읽음)	9.8 GB (가중치 + 게이트)
INT8 (quanto, weight-only, LLM 타워)	transformers + MPS	5.86 GB (-38%)	2.31	1.77 s	bf16과 동일 수준으로 정확함	메모리 내 (디스크 저장 없음)

정밀도	런타임	디바이스 메모리 (MPS 할당)	디코드 tok/s	TTFT	품질 (합성 테스트 이미지)	온디스크
INT4 (quanto, weight-only, LLM 타워)	transformers + MPS	4.15 GB (~56%)	0.47	9.91 s	정확함(표현만 다르게 재구성, 내용은 정확)	메모리 내 (디스크 저장 없음)
INT4 (커뮤니티 MLX 체크포인트)	mlx-vlm	로드 실패	측정 불가	측정 불가	측정 불가	2.9 GB

bf16 모델의 로드 시간은 약 12초였다. MLX 4비트 커뮤니티 체크포인트는 `mlx-vlm` 메인라인이 커스텀 아키텍처 `mage_v1`을 등록하고 있지 않아 (“Model type `mage_v1` not supported”) 로드 자체가 실패했다. 그래서 우리는 이 체크포인트의 속도를 측정하지 못했고, 대신 달성 가능한 양자화 크기의 참고값으로 온디스크 2.9GB만 보고한다. 속도를 확인하려면 `mlx-vlm`에 `mage_v1` 아키텍처를 직접 포팅하는 작업이 필요하다.

4.2 그림: 메모리와 처리량

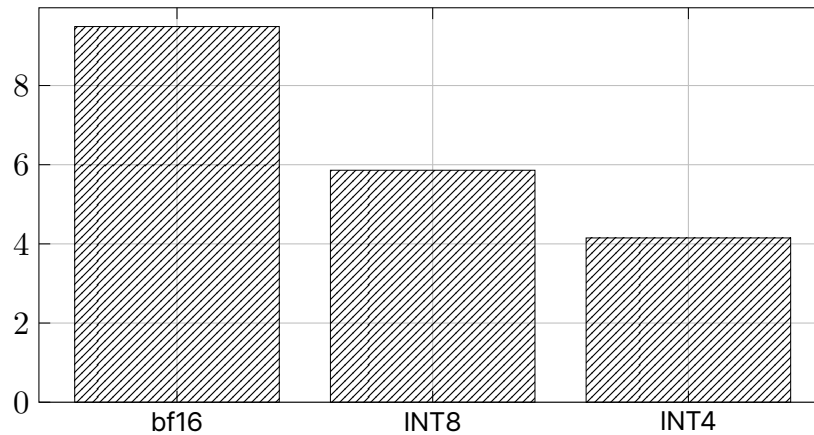


그림 1: 정밀도별 디바이스 메모리

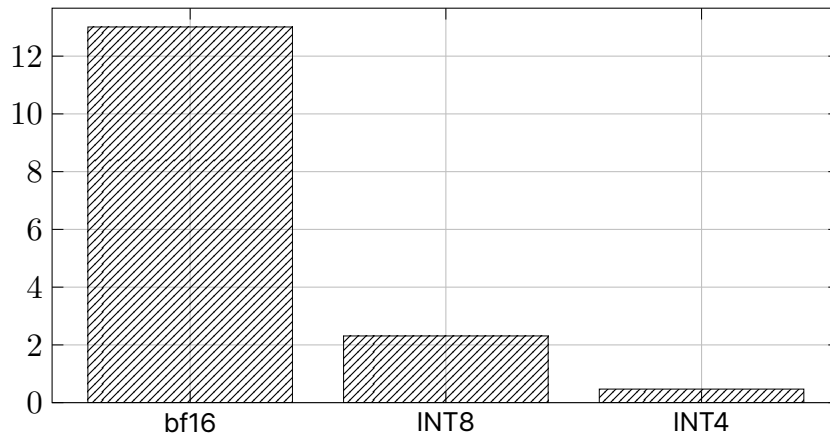


그림 2: 정밀도별 디코드 처리량

두 그림을 겹쳐 보면 이 논문의 핵심이 한눈에 들어온다. 왼쪽 축(메모리)은 정밀도가 낮아질수록 단조롭게 줄어들고, 오른쪽

축(처리량)은 정밀도가 낮아질수록 단조롭게 줄어든다. 두 곡선이 같은 방향이 아니라 서로 어긋나며 움직인다는 것, 그것이 4절에서 다룬 해석의 출발점이다.

5 해석: 메모리는 줄고 속도도 줄어드는 이중성

표준적인 기대는 “양자화하면 더 작고 더 빠르다”는 것이다. 이 실측은 그 기대의 절반만 확인하고 절반은 정면으로 반박한다. 메모리 절감은 기대대로 작동했다. bf16 9.49GB에서 INT4 4.15GB까지, 56%의 디바이스 메모리를 되찾았다. 이는 같은 48GB 통합 메모리 Mac에서 다른 작업(예를 들어 더 긴 컨텍스트, 혹은 동시에 돌아가는 다른 프로세스)에 쓸 여유를 만들어주고, 이론적으로는 메모리가 더 작은 하드웨어에도 이 모델을 태울 수 있다는 뜻이다.

그런데 디코드 처리량은 정반대로 움직였다. bf16 13.01 tok/s에서 INT8 2.31 tok/s로 82% 느려지고, INT4 0.47 tok/s로는 bf16 대비 96% 느려졌다. 같은 방향으로 가야 할 두 지표가 반대로 갈라진 이유는 하드웨어가 아니라 **커널**에 있다. optimum-quanto의 weight-only 양자화는 가중치를 저비트로 저장하지만, 행렬곱을 수행하는 순간에는 그 값을 다시 bf16/fp32로 **즉석에서 역양자화(dequantize)**한 뒤에 계산한다. Apple Metal(MPS) 경로에는 이 저비트 표현을 직접 받아 계산하는 융합(fused) 저비트 행렬곱 커널이 없다. 그래서 메모리 대역폭은 줄어들지만, 매 순전파마다 역양자화라는 추가 연산 비용이 붙는다.

이 비용은 디코드에서 특히 크게 나타난다. 디코드는 토큰 하나를 만들 때마다 전체 언어모델 가중치를 다시 한 번 통과하는 구조라, 128개 토큰을 만드는 동안 역양자화 오버헤드가 128번 반복해서 청구된다. 반면 TTFT는 419개 프롬프트 토큰을 한 번의 배치 연산으로 처리하는 프리필 단계라 오버헤드가 훨씬 덜 누적된다. 실제로 TTFT는 bf16 2.94초에서 INT8 1.77초로 오히려 더 짧아졌는데, 이는 배치가 큰 연산에서는 축소된 메모리 대역폭 이득이 역양자화 비용을 앞섰기 때문으로 보인다. 하지만 INT4로 가면 TTFT마저 9.91초로 bf16보다 훨씬 느려진다. 4비트 압축은 저장 밀도는 더 높지만 역양자화·비트 언패킹 비용도 더 크고, 이 비용이 어느 지점부터는 대역폭 이득을 역전시킨다는 뜻이다. 우리는 이 역전이 정확히 어느 배치 크기·시퀀스 길이에서 발생하는지 프로파일링하지는 않았다. 이는 5절에서 명시하는 한계다.

이 실측이 주는 공학적 교훈은 명확하다. **메모리와 속도는 서로 다른 축이다.** 저비트 가중치는 그 자체로 속도를 보장하지 않는다. 엣지 양자화가 실제로 속도 이득까지 내려면, 저비트 표현을 직접 받아 계산하는 융합 커널을 가진 런타임이 필요하다. MLX, llama.cpp/GGUF 계열이 이런 런타임의 대표적인 예다. 이 실험에서 우리가 시도한 커뮤니티 MLX INT4 체크포인트가 바로 그 경로였지만, `mage_v1`이라는 커스텀 아키텍처가 아직 `mlx-vlm`에 등록되어 있지 않아 로드 자체가 막혔다(3.1절). 즉 “더 작고 더 빠른” 조합을 실제로 얻으려면, quanto 같은 weight-only 경로가 아니라 **아키텍처 포팅이 완료된 융합 커널 런타임**이 필요하다는 것이 이 논문이 데이터로 보여주는 결론이다.

6 한계와 스코프

이 결과를 일반화하기 전에 분명히 밝혀야 할 경계가 몇 가지 있다.

스트리밍 비디오 경로는 스코프 밖이다. DCVC-RT 신경 코덱의 빠른 경로는 CUDA .cu 확장에 묶여 있고, Apple Metal에서는 그대로 실행되지 않는다. 이는 모델의 결함이나 하드웨어 이식성의 경계이며, 우리는 이 경계를 숨기지 않고 이 논문의 스코프에서 명시적으로 제외했다. 이미지/프레임 단위 이해는 이 코덱을 거치지 않으므로 이 논문의 측정과 무관하게 Metal에서 온전히 동작한다.

MLX 네이티브 경로는 아키텍처 포팅이 필요하다. “작고도 빠른” 조합을 같은 하드웨어에서 얻으려면 `mlx-vlm`에 `mage_v1` 아키텍처를 등록하는 포팅 작업이 필요하다. 이 논문은 그 포팅을 완료하지 못했고, 그래서 MLX 경로의 속도는 측정하지 못했다(온디스크 크기 2.9GB만 보고).

정확도 평가는 일반화되지 않는다. 품질 판정은 우리가 직접 만든 합성 이미지 한 장, 단일 프롬프트에 대한 정성적 대조다. 자동화된 스코어링도 아니고, 다양한 이미지·프롬프트에 대한 통계적 정확도도 아니다. “세 정밀도 모두 이 특정 테스트에서 정확했다”는 결과를 “이 모델은 양자화해도 항상 정확하다”로 일반화해서는 안 된다.

저자 보고 수치는 우리 실측과 다른 조건이다. 1.3절에 인용한 시각 토큰 절감률, 속도 향상, Food-101/ImageNet/SoccerNet/OVO-Bench 수치는 8×B200 데이터센터 노드에서 저자들이 측정한 것이며, 우리의 소비자용 Mac 실측과는 하드웨어·데이터셋·평가 방법이 전혀 다르다. 두 수치 집합을 직접 비교하거나 하나를 다른 하나의 증거로 쓰지 않는다.

7 데이터와 재현

- **모델:** microsoft/Mage-VL (Hugging Face). 라이선스 Apache-2.0. Mage-ViT 비전 인코더 + Qwen3-4B-Instruct-2507 언어 백본. 로드된 파라미터 수 실측 4.742B. StreamMind 이벤트 게이트 가중치 1.07GB. DCVC-RT

신경 비디오 코덱 (C++/CUDA 확장 포함).

- **하드웨어:** Apple M4 Pro, 통합 메모리 48GB, macOS, arm64.
- **소프트웨어:** transformers 5.12, torch 2.12, 디바이스 MPS. 양자화는 optimum-quanto(weight-only, language_model 서브모듈에만 적용, 시각 인코더는 bf16 유지). MLX 시도는 mlx-vlm을 사용했다.
- **테스트 입력:** 합성(synthetic) 768×512 “역 전광판” 이미지. 정답은 제목 “CENTRAL STATION - DEPARTURES”, 3행(“14:05/PLATFORM 2/ON TIME”, “14:20/PLATFORM 5/DELAYED”, “14:35/PLATFORM 1/BOARDING”), 기차 픽토그램, 빨간 STOP 표지판으로 우리가 직접 정의했다. 실사진이 아니다.
- **프롬프트/생성 설정:** “Describe this image in detail...” 형태의 상세 서술 요청. 프롬프트 길이(비전 토큰 포함) 419 토큰, 고정 생성 길이 128토큰.
- **측정 방법:** 디바이스 메모리는 MPS 할당량, 디코드 tok/s는 128토큰 생성 구간 평균, TTFT는 419토큰 프리필 완료 후 첫 출력 토큰까지의 지연. 품질은 정답 요소와의 정성적 대조.
- **재현 시 주의:** MLX 4비트 경로는 mlx-vlm 메인라인에 mage_vl 아키텍처가 등록되어 있지 않아 그대로 로드되지 않는다 (“Model type mage_vl not supported”). 재현하려면 아키텍처 포팅이 선행되어야 한다.

8 참고문헌

1. Hugging Face 모델 카드: microsoft/Mage-VL (Apache-2.0), huggingface.co/microsoft/Mage-VL
2. Qwen/Qwen3-4B-Instruct-2507 모델 카드, huggingface.co/Qwen
3. Hugging Face transformers 라이브러리 문서, huggingface.co/docs/transformers
4. optimum-quanto (weight-only 양자화), github.com/huggingface/optimum-quanto
5. mlx-vlm (Apple MLX 비전-언어모델 런타임), github.com/Blaizzy/mlx-vlm
6. Apache License 2.0 전문, apache.org/licenses/LICENSE-2.0